

Smart Stethoscope

An IoT-Based Device for Remote Heartbeat Monitoring

Project Lead: Avani Sunil Deshpande

Guide: Mrs. Leena Jadhav

Dnyandeep English Medium School

Submission for INSPIRE Award

2021–22

Dnyandeep English Medium School

Inspire Award Exhibition 2021-22

Name of Project

Smart Stethoscope



Aim

To record heartbeat sound efficiently and economically.

Materials

Node MCU Microcontroller (ESP8266), HW484 Microphone, Jumper Wires, a LED bulb

How it works

- The code that runs through the Microcontroller, allows the microphone to record the throbbing of the heart.
- The microphone records sound for 20 seconds, and then transfers the sound to a thingspeak server.
- The server can then regulate and transfer the audio clip to other stakeholders involved.
- The doctors can then hear the clip with sound magnifiers or speakers with a relative ease.

Uses of Project

- Helps the patients living in rural areas to have access to primary healthcare.
- Reduces doctors time in the cabin, and increases efficiency in the clinic.
- Is a good to have device for a second opinion over health.

Student's name
Avani Sunil Deshpande

Teacher's name
Leena Jadhav

Contents

1	Abstract	4
2	Introduction	4
3	Problem Statement	4
4	Objectives	4
5	Materials Required	5
6	Block Diagram and Working	5
7	Hardware Setup	6
8	Software Design	6
8.1	Recording Parameters	7
8.2	Signal Processing Method	7
8.2.1	Normalisation	7
8.2.2	Simple Moving Average (SMA) Filter	7
8.2.3	Threshold-Based Beat Detection	7
8.2.4	BPM Calculation	7
8.3	ThingSpeak Data Fields	8
9	Results and Testing	8
10	Applications	10
11	Limitations	11
12	Future Scope	11
13	Conclusion	11
A	Source Code	12

1 Abstract

Healthcare access remains limited in many rural parts of India. A conventional stethoscope works well in a clinic, but the recorded heart sound cannot be shared easily with a doctor at another location. This project presents a **Smart Stethoscope**: a low-cost IoT device built using a NodeMCU (ESP8266) microcontroller and an HW-484 microphone module. The device records heartbeat sound for 20 seconds, processes the signal to estimate heart rate, and uploads the results to a ThingSpeak cloud server over Wi-Fi. Doctors or health workers can then view the data remotely and use it for preliminary analysis. The total cost of the prototype is much lower than commercial digital stethoscopes, making it suitable for schools, primary health centres, and telemedicine camps.

2 Introduction

The stethoscope is one of the most common tools used by doctors to listen to heart and lung sounds. However, in villages and small towns, specialist doctors are not always available on the same day. Patients often travel long distances for a basic check-up. At the same time, low-cost microcontrollers and cloud platforms have made it possible for students to build useful healthcare devices in the school laboratory.

Our project converts an ordinary stethoscope into a smart device. Heart sound is picked up by a microphone, digitised by the NodeMCU, analysed using simple signal processing, and sent to the internet. This idea was developed for the **INSPIRE Award Exhibition 2021–22** at Dnyandeep English Medium School.

3 Problem Statement

Many rural patients do not get timely cardiac screening because:

[noitemsep]

- qualified doctors are available only at district hospitals;
- digital stethoscopes and ECG machines are expensive;
- heart sounds cannot be stored or forwarded using a normal stethoscope.

There is a need for an economical device that can capture heartbeat sound, calculate basic parameters such as beats per minute (BPM), and share the reading with a doctor through the internet.

4 Objectives

[label=4.]

1. To design a low-cost heartbeat recording system using easily available components.
2. To record heart sound for 20 seconds through a microphone attached to a stethoscope chest piece.

3. To filter noisy sensor data and detect individual heartbeats.
4. To calculate heart rate (BPM) from the recorded signal.
5. To upload results to a ThingSpeak server using Wi-Fi.
6. To demonstrate the use of the device for rural healthcare and telemedicine.

5 Materials Required

Table 1: Components used in the Smart Stethoscope prototype

Component	Purpose
NodeMCU (ESP8266)	Main microcontroller with built-in Wi-Fi
HW-484 microphone module	Converts heart sound into electrical signal
Stethoscope chest piece	Collects acoustic signal from the chest
Jumper wires	Connections between modules
LED bulb	Shows recording / system status
Breadboard	Prototype assembly
USB cable	Power supply and programming
Wi-Fi network	Internet connectivity
ThingSpeak account	Cloud storage and visualisation

6 Block Diagram and Working

The working of the Smart Stethoscope can be explained in the following steps:

[label=6.]

1. The patient places the stethoscope on the chest.
2. The HW-484 microphone converts sound into an analog voltage.
3. The NodeMCU reads the signal through analog pin A0.
4. The device records samples for 20 seconds and stores them in memory.
5. A Simple Moving Average (SMA) filter removes high-frequency noise.
6. Heartbeats are detected when the normalised signal crosses a threshold of 0.5.
7. BPM is calculated from the number of detected beats.
8. Results are uploaded to ThingSpeak fields for remote viewing.

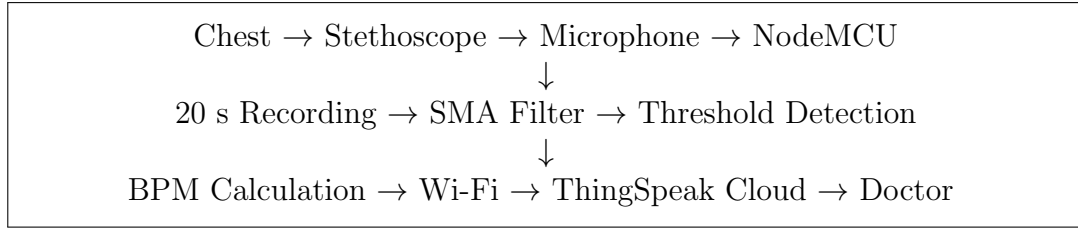


Figure 1: Functional block diagram of the Smart Stethoscope system

7 Hardware Setup

The microphone module was fixed near the stethoscope chest piece so that heart vibrations could be picked up clearly. The NodeMCU was mounted on a breadboard along with an LED indicator. During recording, the LED glows so that the user knows the device is active.

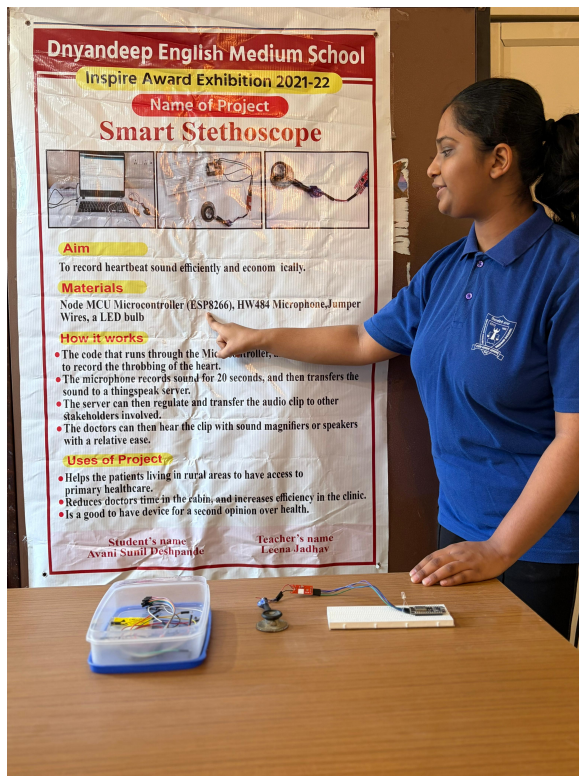


Figure 2: Successful demonstration of the INSPIRE Award project to school faculty and peers.

8 Software Design

The firmware was written in C++ using the Arduino framework for ESP8266. Two libraries were used:

[noitemsep]

- `ESP8266WiFi.h` for Wi-Fi connection;
- `ThingSpeak.h` for uploading data to the cloud server.

8.1 Recording Parameters

[noitemsep]

- Sampling rate: 50 samples per second
- Recording duration: 20 seconds
- Total samples stored: 1000
- Analog input pin: A0

8.2 Signal Processing Method

Heart sound from the microphone contains both useful beats and background noise. To analyse it properly, the following method was used:

8.2.1 Normalisation

Each raw reading from the ADC (0 to 1023) is converted to a value between 0 and 1:

$$x[n] = \frac{\text{raw}[n]}{1023}$$

8.2.2 Simple Moving Average (SMA) Filter

A moving average of window size 10 was applied to smooth the signal:

$$y[n] = \frac{1}{N} \sum_{k=n-N+1}^n x[k]$$

This step reduces sudden spikes caused by noise while keeping the overall heartbeat pattern.

8.2.3 Threshold-Based Beat Detection

A threshold line was fixed at amplitude 0.5. Whenever the normalised raw signal crossed this level, it was treated as a possible heartbeat. A minimum gap of 25 samples (0.5 seconds) was kept between two beats so that the same heartbeat was not counted twice.

8.2.4 BPM Calculation

If B beats are detected in 20 seconds, then:

$$\text{BPM} = \frac{B \times 60}{20} = 3B$$

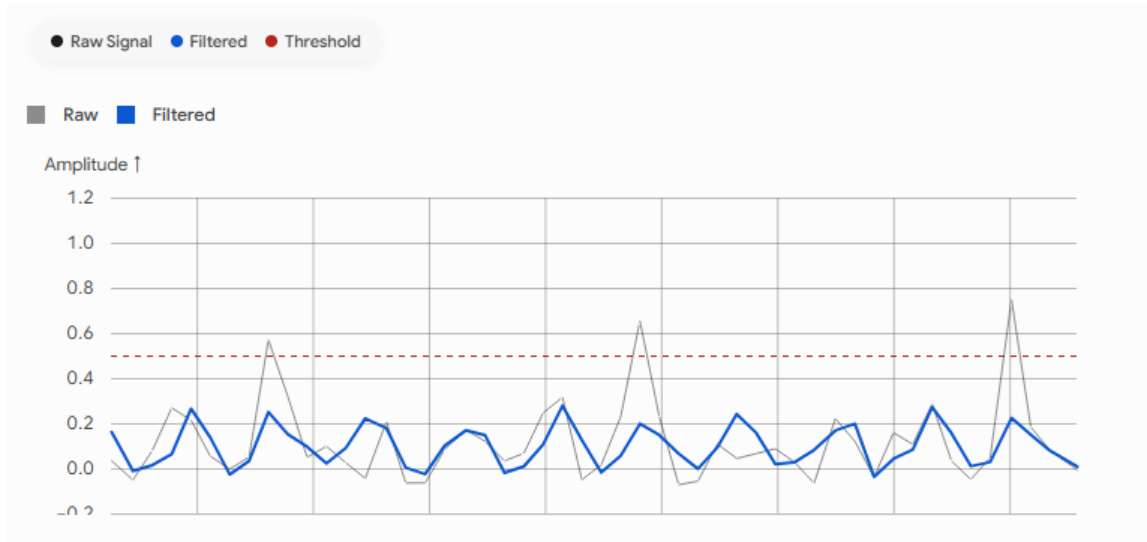


Figure 3: Raw signal, filtered signal, and threshold line used for heartbeat analysis

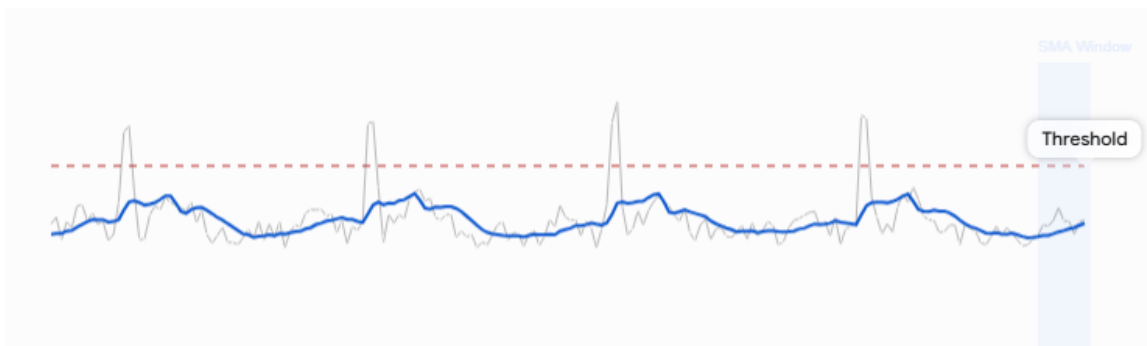


Figure 4: Validation of peak detection using SMA filtering and threshold crossing

8.3 ThingSpeak Data Fields

After each recording cycle, the following values were uploaded:

Table 2: ThingSpeak channel fields

Field	Data sent
1	Heart rate (BPM)
2	Number of beats detected
3	Average microphone reading
4	Peak microphone reading

9 Results and Testing

Initial testing was carried out using the Wokwi online simulator and later on the physical prototype. The serial monitor showed Wi-Fi connection status, sensor readings, beat count, BPM, and ThingSpeak upload confirmation.

```
Connecting to Network: YOUR_WIFI_SSID . . .  
WiFi Connected Successfully!  
IP Address: 192.168.1.104  
  
Microphone Sensor Value: 512  
ThingSpeak Channel update successful.  
(Waiting for next reading...)  
  
Microphone Sensor Value: 515  
ThingSpeak Channel update successful.  
(Waiting for next reading...)  
  
Microphone Sensor Value: 680  
ThingSpeak Channel update successful.  
(Waiting for next reading...)  
  
Microphone Sensor Value: 508  
ThingSpeak Channel update successful.  
(Waiting for next reading...)  
  
Microphone Sensor Value: 712  
ThingSpeak Channel update successful.  
(Waiting for next reading...)  
  
Simulation Complete.  
  
...Program finished with exit code 0  
Press ENTER to exit console. □
```

Figure 5: Serial monitor output showing microphone readings and successful ThingSpeak updates

Sample observations from testing:

[noitemsep]

- The device connected successfully to Wi-Fi and obtained an IP address.
- Microphone readings changed when the stethoscope was placed on the chest.
- Higher amplitude values were observed during stronger heart sounds.
- BPM values were obtained after threshold-based beat counting.
- Data upload to ThingSpeak returned HTTP code 200 (success).



Figure 6: Receiving the INSPIRE Award for the Smart Stethoscope project.

10 Applications

[noitemsep]

- **Rural healthcare:** Patients in villages can get a basic heart check without travelling to a city hospital.
- **Telemedicine:** Recorded BPM and signal data can be shared with a doctor for review.
- **School health camps:** Useful for quick screening during medical camps.
- **Second opinion:** Helps doctors compare readings over time.
- **Science education:** Demonstrates practical use of IoT, sensors, and cloud computing.

11 Limitations

[noitemsep]

- The ESP8266 analog pin accepts only 0–1 V, so the microphone output must stay within safe limits.
- Background noise can affect beat detection if the threshold is not tuned properly.
- ThingSpeak free account allows one update every 15 seconds, so full audio waveform upload is not possible in a single packet.
- This device gives a preliminary reading and cannot replace professional medical diagnosis.

12 Future Scope

[noitemsep]

- Add a small display to show BPM directly on the device.
- Improve filtering using band-pass filters for heart sound frequencies.
- Store and replay the 20-second audio clip on a mobile app.
- Use a rechargeable battery for portable field use.
- Send alert messages when BPM goes outside the normal range.

13 Conclusion

The Smart Stethoscope project shows that a useful healthcare tool can be built using low-cost electronics and free cloud services. By combining a stethoscope, microphone, NodeMCU, and ThingSpeak server, we were able to record heart sound, process it, estimate BPM, and share the result online. The project supports the goal of affordable primary healthcare in rural areas and demonstrates how school-level innovation can address real social problems. With further improvement, this device can become a helpful aid for doctors, health workers, and telemedicine programmes.

Acknowledgement

We thank Mrs. Leena Jadhav for her guidance throughout this project. We also thank Dnyandeep English Medium School for providing the opportunity to present this work under the INSPIRE Award scheme.

References

- [1] Espressif Systems, “ESP8266 Technical Reference,” <https://www.espressif.com/>
- [2] MathWorks, “ThingSpeak IoT Platform Documentation,” <https://thingspeak.com/>
- [3] Department of Science and Technology, “INSPIRE Awards – MANAK,” Government of India.
- [4] World Health Organization, “Telemedicine: Opportunities and Developments in Member States.”
- [5] Keyes / HW-484 Microphone Sensor Module Datasheet.

A Source Code

The main firmware file `heartbeat_read.cpp` is given below.

```
1 #include <ESP8266WiFi.h>
2 #include <ThingSpeak.h>
3
4 const char* ssid = "YOUR_WIFI_SSID";
5 const char* password = "YOUR_WIFI_PASSWORD";
6
7 unsigned long channelID = 1234567;
8 const char* writeKey = "YOUR_WRITE_API_KEY";
9
10 const int MIC_PIN = A0;
11 const int LED_PIN = LED_BUILTIN;
12
13 // record 20 sec like in our project poster
14 const int SAMPLE_RATE = 50;
15 const int RECORD_SEC = 20;
16 const int BUFFER_SIZE = SAMPLE_RATE * RECORD_SEC;
17 const int SAMPLE_MS = 1000 / SAMPLE_RATE;
18
19 // simple moving average window (same idea as our validation
    graph)
20 const int SMA_WINDOW = 10;
21
22 // normalized amplitude threshold - red dashed line in our chart
    was at 0.5
23 const float THRESHOLD = 0.5;
24
25 // don't count two beats too close together
26 const int MIN_BEAT_GAP = 25;
27
28 uint16_t rawSamples[BUFFER_SIZE];
29 float filteredSamples[BUFFER_SIZE];
30
31 WiFiClient client;
```

```

32
33 void connectWiFi() {
34     if (WiFi.status() == WL_CONNECTED) {
35         return;
36     }
37
38     Serial.println("WiFi lost, reconnecting...");
39     WiFi.disconnect();
40     WiFi.begin(ssid, password);
41
42     while (WiFi.status() != WL_CONNECTED) {
43         delay(500);
44         Serial.print(".");
45     }
46
47     Serial.println("");
48     Serial.println("WiFi Connected Successfully!");
49     Serial.print("IP Address: ");
50     Serial.println(WiFi.localIP());
51 }
52
53 void recordHeartbeat() {
54     Serial.println("Recording 20 sec - hold stethoscope on chest");
55     digitalWrite(LED_PIN, LOW);
56
57     for (int i = 0; i < BUFFER_SIZE; i++) {
58         rawSamples[i] = analogRead(MIC_PIN);
59         delay(SAMPLE_MS);
60     }
61
62     digitalWrite(LED_PIN, HIGH);
63     Serial.println("Recording finished");
64 }
65
66 void applySMA() {
67     for (int i = 0; i < BUFFER_SIZE; i++) {
68         long sum = 0;
69         int count = 0;
70
71         for (int j = i - SMA_WINDOW + 1; j <= i; j++) {
72             if (j < 0) {
73                 continue;
74             }
75             sum += rawSamples[j];
76             count++;
77         }
78
79         filteredSamples[i] = (sum / (float)count) / 1023.0;
80     }
81 }
82

```

```

83 int countBeats(int& maxRaw, int& avgRaw) {
84     long sum = 0;
85     maxRaw = 0;
86
87     for (int i = 0; i < BUFFER_SIZE; i++) {
88         sum += rawSamples[i];
89         if (rawSamples[i] > maxRaw) {
90             maxRaw = rawSamples[i];
91         }
92     }
93
94     avgRaw = sum / BUFFER_SIZE;
95
96     int beats = 0;
97     int lastBeat = -MIN_BEAT_GAP;
98     bool aboveThreshold = false;
99
100    for (int i = 0; i < BUFFER_SIZE; i++) {
101        float rawNorm = rawSamples[i] / 1023.0;
102
103        if (!aboveThreshold && rawNorm > THRESHOLD && (i - lastBeat)
104            >= MIN_BEAT_GAP) {
105            beats++;
106            lastBeat = i;
107            aboveThreshold = true;
108        }
109
110        if (rawNorm < THRESHOLD - 0.05) {
111            aboveThreshold = false;
112        }
113    }
114
115    return beats;
116 }
117
118 void setup() {
119     Serial.begin(115200);
120     delay(100);
121
122     pinMode(MIC_PIN, INPUT);
123     pinMode(LED_PIN, OUTPUT);
124     digitalWrite(LED_PIN, HIGH);
125
126     Serial.println();
127     Serial.print("Connecting to Network: ");
128     Serial.println(ssid);
129
130     connectWiFi();
131     ThingSpeak.begin(client);
132 }

```

```

133 void loop() {
134     connectWiFi();
135
136     recordHeartbeat();
137     applySMA();
138
139     int maxRaw, avgRaw;
140     int beats = countBeats(maxRaw, avgRaw);
141     int bpm = 0;
142
143     if (beats >= 1) {
144         bpm = (beats * 60) / RECORD_SEC;
145     }
146
147     Serial.print("Microphone Sensor Value: ");
148     Serial.println(avgRaw);
149     Serial.print("Beats detected: ");
150     Serial.println(beats);
151     Serial.print("Heart rate (BPM): ");
152     Serial.println(bpm);
153     Serial.print("Peak amplitude: ");
154     Serial.println(maxRaw);
155
156     // send results to thingspeak for doctor to check later
157     ThingSpeak.setField(1, bpm);
158     ThingSpeak.setField(2, beats);
159     ThingSpeak.setField(3, avgRaw);
160     ThingSpeak.setField(4, maxRaw);
161
162     int httpCode = ThingSpeak.writeFields(channelID, writeKey);
163
164     if (httpCode == 200) {
165         Serial.println("ThingSpeak Channel update successful.");
166     } else {
167         Serial.print("Problem updating channel. HTTP error code: ");
168         Serial.println(httpCode);
169     }
170
171     Serial.println("(Waiting for next reading...)");
172     delay(15000);
173 }

```